Branch Prediction Unit (BPU)

Exploratory Project Report

Group Members	Adityaa Mehra (24095005) Pratyush L Daddi (24095080) K.S. Sethurathnam (2409139)
Academic Year	2025–2026
Topic	Design and Simulation of a Tournament Branch Prediction Unit for a RISC-V Pipelined Processor

Abstract

Branch prediction is a critical technique in modern pipelined processors for mitigating the performance penalty of control hazards caused by branch and jump instructions. This report details the design, RTL implementation in Verilog, and simulation of a Tournament Branch Prediction Unit (BPU) integrated into a 5-stage RISC-V pipelined processor. The architecture combines a local history predictor with a global history predictor, selected dynamically by a choice predictor. A Branch Target Buffer (BTB) with full-tag address validation resolves the aliasing problem that arises from indexing with partial PC bits. Performance is benchmarked against baseline, Gshare, and Perceptron predictors using the gem5 architectural simulator, demonstrating up to 5.6× speedup on loop-heavy workloads and 2× on data-dependent branch patterns.

Contents

Chapter 1

Introduction

1.1 Motivation

Modern processors use deep instruction pipelines to sustain high clock frequencies and instruction throughput. Pipelining overlaps the execution of multiple instructions simultaneously, enabling steady-state throughput of one instruction per clock cycle. However, this overlap introduces the *control hazard* problem: when a branch instruction is encountered, the processor does not yet know which path of execution will be taken. Naively stalling the pipeline until the branch outcome is resolved wastes several cycles per branch, severely degrading performance.

A Branch Prediction Unit (BPU) guesses the outcome of a branch before it is decoded or executed, allowing the processor to continue fetching along the predicted path speculatively. A correct prediction incurs no penalty; an incorrect one triggers a pipeline flush equal in cost to the pipeline depth.

1.2 Objectives

1. Understand the architecture of a tournament predictor combining local and global prediction strategies.
2. Design and implement the full BPU in synthesisable Verilog RTL, including the BTB and aliasing mitigation logic.
3. Simulate and verify the design using custom RISC-V assembly programs.
4. Benchmark the tournament predictor against Baseline, Gshare, and Perceptron variants using the gem5 simulator.
5. Analyse the trade-offs between prediction accuracy, IPC, and hardware cost.

1.3 Report Structure

Chapter 2 covers the 5-stage pipeline and control hazards. Chapter 3 describes the tournament BPU architecture in detail. Chapter 4 addresses the aliasing problem in the BTB and its hardware mitigation. Chapter 5 presents the actual Verilog RTL from the repository. Chapter 6 analyses the gem5 simulation results. Chapter 7 concludes with future scope.

Chapter 2

Background: Pipelining and Control Hazards

2.1 The 5-Stage Pipeline

The processor considered in this project is a classic 5-stage in-order pipeline:

Stage	Abbr.	Function
1	IF	Instruction Fetch — reads the instruction from memory using the PC.
2	ID	Instruction Decode — decodes opcode, reads register file, identifies branch type.
3	EX	Execute — performs ALU operations; resolves branch condition and target address.
4	MEM	Memory Access — reads or writes data memory.
5	WB	Write Back — writes results to the register file.

Under ideal conditions the pipeline sustains a throughput of one instruction per clock cycle after the initial 5-cycle fill latency. The first instruction completes only after the 5th clock edge; every edge thereafter retires one instruction.

2.2 Control Hazards

A **control hazard** arises because a branch instruction is fetched in IF but its condition is only resolved in EX, three cycles later. During those three intervening cycles the pipeline has already fetched up to three instructions from the wrong path. On resolution, those instructions are *flushed* (converted to NOPs) and execution restarts from the correct target, wasting 2–3 cycles per misprediction.

The penalty is proportional to pipeline depth — in modern out-of-order processors with 15–20 stages, a single misprediction can cost 15+ cycles.

2.2.1 Code Example Producing a Control Hazard

```c
for (int i = 0; i < t; i++) {
    int a;
    scanf("%d", &a);
    if (a % 2 == 0)          // branch: taken -> printf("even")
        printf("even");
    else                     // not-taken -> printf("odd")
        printf("odd");
}
```

Listing 2.1: C loop producing a control hazard on the branch instruction

The `if` statement compiles to a conditional branch. At fetch time the processor cannot know whether `a % 2 == 0` is true, so it must either stall or speculate.

2.3 Branch Prediction as the Solution

Branch prediction estimates the outcome (taken / not-taken) and target address of a branch *before* it reaches the EX stage, enabling continuous instruction fetch without stalling. The predictor operates in IF, before decode — meaning it evaluates every instruction speculatively, since it does not yet know whether the fetched instruction is even a branch.

2.3.1 Prediction Accuracy and the Hit Ratio

$$\text{Hit Ratio} = \frac{\text{Correct Predictions (taken + not-taken)}}{\text{Total Predictions Made}}$$

Even a small improvement in hit ratio translates to significant IPC gains in branch-heavy workloads.

2.3.2 Taxonomy of Branch Predictors

- **Static:** Always-taken or always-not-taken. No runtime state.
- **Dynamic:** Maintain saturating counters indexed by PC history (bimodal, local, Gshare).
- **Hybrid / Tournament:** Combine multiple sub-predictors; a choice mechanism selects the better prediction per branch.
- **Neural:** Perceptron-based weight vectors capturing long-range history correlations at high hardware cost.

Chapter 3

Tournament Branch Predictor Architecture

3.1 Overview

The **tournament predictor** is a hybrid architecture that dynamically selects between a **Local Predictor** and a **Global Predictor** on a per-branch basis. A **Choice Predictor** (a 2-bit saturating counter table) learns, for each branch context, whether the local or global sub-predictor has historically been more accurate. This design was popularised in the Alpha 21264 and Intel Pentium-class processors and forms the reference architecture for this project.

3.2 Local Predictor: `TwoLevelLocalPred`

The local predictor is a two-level adaptive predictor:

- **Local History Table (LHT)** — size $2^{\texttt{PC\_BITS}} \times \texttt{HIST\_BITS}$. Indexed by PC[PC_BITS+1:2] (word-aligned lower bits). Each entry stores a shift register of the last `HIST_BITS` outcomes for that specific branch PC.
- **Pattern History Table (PHT)** — size $2^{\texttt{HIST\_BITS}}$ entries of 2-bit saturating counters (`STRONGLY_NOT_TAKEN`=`2'b00`, `WEAKLY_NOT_TAKEN`=`2'b01`, `WEAKLY_TAKEN`=`2'b10`, `STRONGLY_TAKEN`=`2'b11`). Indexed by the local history pattern for this branch.

The local predictor excels at capturing per-branch periodic or loop-like behaviour (e.g., a `for` loop that is taken $N-1$ times then not-taken once).

3.2.1 Update Rule (PHT)

On branch resolution the PHT counter at the *old* history index is updated:

Current State	Actual Outcome	Next State
STRONGLY_NOT_TAKEN	0	STRONGLY_NOT_TAKEN
STRONGLY_NOT_TAKEN	1	WEAKLY_NOT_TAKEN
WEAKLY_NOT_TAKEN	0	STRONGLY_NOT_TAKEN
WEAKLY_NOT_TAKEN	1	WEAKLY_TAKEN
WEAKLY_TAKEN	0	WEAKLY_NOT_TAKEN
WEAKLY_TAKEN	1	STRONGLY_TAKEN
STRONGLY_TAKEN	0	WEAKLY_TAKEN
STRONGLY_TAKEN	1	STRONGLY_TAKEN

The LHT entry for the branch PC is updated by left-shifting in the actual outcome: `lht[update_idx] <= {update_history_in[HIST_BITS-2:0], branch_taken}`.

3.3 Global Predictor: `GlobalPred`

- **Global History Register (GHR)** — `GHR_BITS`-wide shift register recording the outcomes of the last `GHR_BITS` branches globally (not per-PC).
- **Pattern History Table (PHT)** — $2^{\texttt{PHT\_INDEX\_BITS}}$ entries of 2-bit saturating counters. Indexed by the **XOR** of the PC bits and the GHR:

$$\texttt{xored\_idx} = \texttt{fetch\_pc[PHT\_INDEX\_BITS+1:2]} \oplus \texttt{ghr}$$

This XOR folding (Gshare-style) is the key aliasing-mitigation technique for the prediction tables and is discussed in depth in Chapter 4. The global predictor captures correlations between different branches — for example, a branch whose outcome depends on the results of earlier branches in the same basic-block context.

3.4 Choice Predictor: `ChoicePred`

- **Choice Pattern Table (CPT)** — $2^{\texttt{GHR\_BITS}}$ entries of 2-bit saturating counters, with states `STRONGLY_LOCAL`=`2'b00`, `WEAKLY_LOCAL`=`2'b01`, `WEAKLY_GLOBAL`=`2'b10`, `STRONGLY_GLOBAL`=`2'b11`. Indexed by the GHR at the time of fetch.
- The output bit `use_global_pred` drives the final MUX: counter $\geq$ `WEAKLY_GLOBAL` $\Rightarrow$ select global; otherwise select local.

3.4.1 Choice Predictor Update Rule

local_correct	global_correct	CPT Update
1	0	Decrement (toward `STRONGLY_LOCAL`), saturate at 0
0	1	Increment (toward `STRONGLY_GLOBAL`), saturate at 3
0	0	No change
1	1	No change

3.5 Top-Level Tournament Predictor: `TournamentPredictor`

The `TournamentPredictor` module instantiates all three sub-predictors and implements the final MUX:

$$\texttt{final\_prediction} = \texttt{use\_global\_pred} \; ? \; \texttt{global\_pred\_out} \; : \; \texttt{local\_pred\_out}$$

3.6 Architecture Block Diagram

Figure 3.1 shows the tournament BPU architecture. The Program Counter provides the fetch-time index to both the Local History Table and the XOR logic. The GHR feeds both the Global PHT and the Choice PT. The final MUX is driven by the choice predictor output.

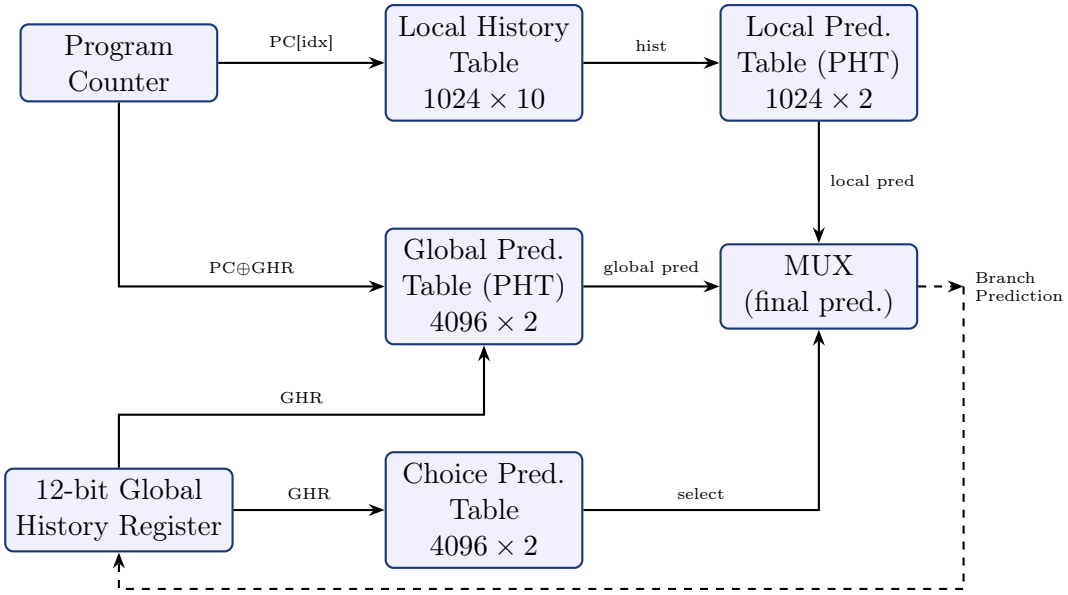

Figure 3.1: Tournament BPU top-level architecture. The Choice Prediction Table dynamically selects between local and global sub-predictors. Resolved branch outcomes feed back to update all tables and shift the GHR.

3.7 Module Hierarchy

Module	Description
`final_bpu`	Top-level wrapper. Instantiates `TournamentPredictor` and `BTB`, computes `next_pc`.
`TournamentPredictor`	Instantiates the three sub-predictors and implements the final MUX.
`TwoLevelLocalPred`	Two-level local predictor (LHT + PHT).
`GlobalPred`	Gshare-style global predictor (GHR + XOR PHT).
`ChoicePred`	Choice/meta predictor (CPT indexed by GHR).
`BTB`	Branch Target Buffer with tag validation.

Chapter 4

Branch Target Buffer and Aliasing

4.1 Role of the Branch Target Buffer

When a branch is predicted *taken*, the processor must also know the **target address** to begin fetching from. The Branch Target Buffer (BTB) is a direct-mapped cache that stores `(tag, target)` pairs for recently seen taken branches. It allows the processor to redirect the PC in the same cycle as the direction prediction.

4.2 The Aliasing Problem

4.2.1 Cause

To limit BTB area, only the lower `PC_BITS` bits of the 32-bit PC (word-aligned, so bits [PC_BITS+1:2]) are used to index the BTB. Because the branch predictor evaluates *every* instruction speculatively in IF, two branch instructions with the same lower-byte PC suffix collide at the same BTB row. Example for 8-bit indexing:

$$\text{PC}_1 = \texttt{32'h7741B621} \;\Rightarrow\; \text{BEQZ},\ \text{Target} = \texttt{32'hB3251374}$$
$$\text{PC}_2 = \texttt{32'h18390521} \;\Rightarrow\; \text{BNEZ},\ \text{Target} = \texttt{32'h17265422}$$

Both share `PC[7:0] = 0x21`, mapping to BTB index 33. Each branch overwrites the other's target entry. This is **aliasing** (collision).

4.2.2 Destructive vs. Constructive Aliasing

- **Destructive aliasing:** Two branches alternately overwrite each other's entry, causing persistent mispredictions for both. This is the harmful case targeted by the mitigation strategies below.
- **Constructive aliasing:** Two branches share a BTB slot but happen to have similar taken/not-taken patterns, so accuracy is not degraded.

4.3 Mitigation 1 — XOR Folding in the Prediction Tables

Instead of using raw PC bits to index the Global PHT and Choice PT, the global predictor XORs the PC index bits with the GHR:

$$\texttt{xored\_idx} = \texttt{fetch\_pc[PHT\_INDEX\_BITS+1:2]} \oplus \{\texttt{0...0},\ \texttt{ghr}\}$$

Since the GHR shifts in a new bit on every branch event, two instructions with the same lower PC bits will almost always produce different indices, dramatically reducing

destructive aliasing in the prediction tables. The update path computes the same XOR using the history value captured at fetch time:

$$\texttt{update\_xored\_idx} = \texttt{update\_pc[PHT\_INDEX\_BITS+1:2]} \oplus \{\texttt{0...0}, \texttt{update\_history\_in}\}$$

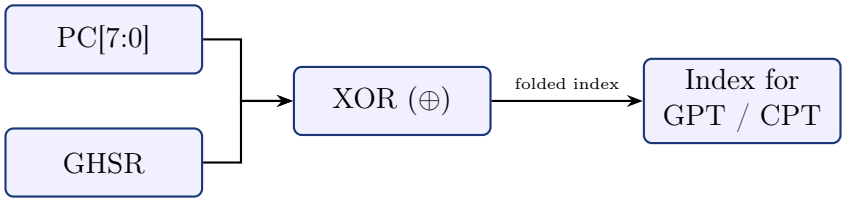

Figure 4.1: Functional address calculation. XOR of PC[7:0] and the Global History Shift Register produces context-sensitive indices that eliminate destructive aliasing in the prediction tables.

4.4 Mitigation 2 — Tag-Based Address Validation in the BTB

Even with XOR-based indexing for the predictors, the BTB itself uses the raw lower PC bits for the array index. Address validation prevents incorrect targets from being supplied to unrelated instructions:

1. The lower `PC_BITS` bits of the incoming PC (bits [PC_BITS+1:2]) index the BTB array row.
2. The stored `tag_array[fetch_idx]` (the full 32-bit PC of the branch that last wrote this row) is compared against the current `fetch_pc`.
3. If `valid[fetch_idx] && tag_array[fetch_idx] == fetch_pc` then `btb_hit = 1` and the stored target is provided.
4. Otherwise `btb_hit = 0` and the processor falls back to PC+4 (not-taken path). No aliased target is propagated.

The BTB only stores entries for *taken* branches (`update_en && branch_taken`), keeping the table populated with useful predictions only.

4.5 Combined Anti-Aliasing Flow

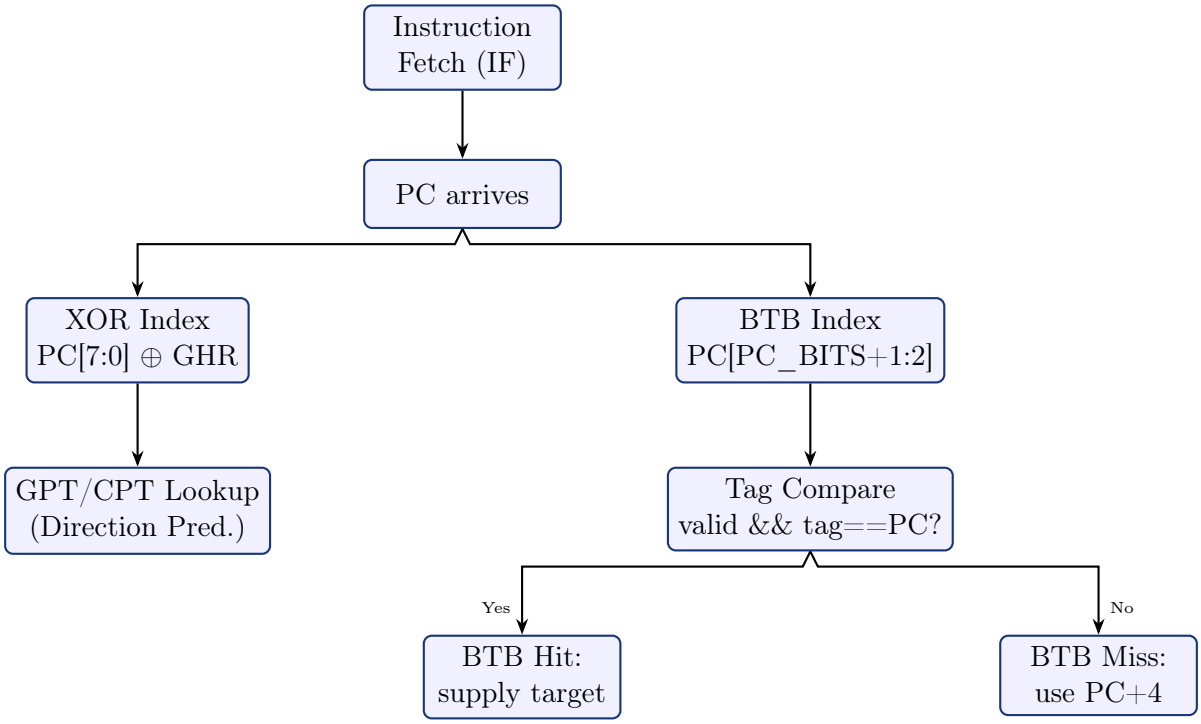

Figure 4.2: Combined aliasing mitigation flow. XOR folding handles the direction predictor tables; tag comparison handles the BTB target selection.

Chapter 5

Simulation and Performance Results

5.1 Simulation Environment

gem5 is an open-source, cycle-accurate architectural simulator supporting RISC-V and a wide range of processor models. In this project gem5 is used to:

- Model the 5-stage RISC-V pipeline with configurable branch prediction.
- Integrate and evaluate each predictor variant by swapping the branch predictor configuration.
- Collect per-run statistics: IPC, direction accuracy, BTB hit rate, and conditional misprediction count.

5.2 Benchmark Programs

5.2.1 Benchmark 1 — Simple Counting Loop (Highly Predictable)

```
PC 0: addi  $t0, $zero, 5    // Counter = 5
PC 1: addi  $t0, $t0,  -1    // Counter--
PC 2: bne   $t0, $zero, 1    // Loop back to PC 1  (taken 4/5 times)
PC 3: beq   $zero, $zero, 3  // Infinite Halt
```

Listing 5.1: Benchmark 1: simple 5-iteration counting loop. The branch at PC 2 is taken 4 out of 5 times in a perfectly regular pattern.

The loop branch is taken in a regular pattern that even a 2-bit bimodal predictor learns quickly. The tournament predictor reaches near-perfect accuracy after one full loop iteration.

5.2.2 Benchmark 2 — Nested Loop with Unpredictable Branch

```
// Initialization
PC  0: addi $t0, $zero, 4    // Outer counter i = 4
PC  1: addi $t2, $zero, 42   // Initial data = 42
// Outer loop
PC  2: beq  $t0, $zero, 13   // If i==0, jump to HALT
PC  3: add  $t1, $zero, $t0  // Inner counter j = i
// Inner loop
PC  4: beq  $t1, $zero, 11   // If j==0, jump to outer decrement
PC  5: andi $t4, $t2,   1    // Check data odd/even
PC  6: beq  $t4, $zero, 8    // *** UNPREDICTABLE BRANCH ***
// Mutate path (odd)
PC  7: addi $t2, $t2,   13   // Mutate data state 1
// Continue inner loop
PC  8: addi $t2, $t2,   -1   // Mutate data state 2 (shifts randomisation)
PC  9: addi $t1, $t1,   -1   // j--
PC 10: beq  $zero, $zero, 4  // Jump back to inner loop
```

```
// Continue outer loop
PC 11: addi $t0, $t0,   -1   // i--
PC 12: beq  $zero, $zero, 2  // Jump back to outer loop
// End
PC 13: beq  $zero, $zero, 13 // Infinite Halt loop
```

Listing 5.2: Benchmark 2: nested loop with a data-dependent "unpredictable" branch at PC 6. The data state evolves chaotically via the +13 and -1 mutations, preventing simple local history from learning it.

The branch at PC 6 is conditioned on whether the evolving data register `$t2` is odd or even after each mutation step. The combination of `+13` and `-1` produces a chaotic pattern. This workload stresses the global and choice prediction components of the tournament predictor, since global history correlations (outer vs. inner loop context) provide more information than per-branch local history alone.

5.3 gem5 Results

Table 5.1: Comparative performance of branch predictors on the 5-stage RISC-V pipeline (gem5 simulation)

Predictor	IPC	Dir. Acc. (%)	Cond. Mispreds	BTB Hit (%)	Est. HW Budget	Logic Complexity
Baseline (Default)	1.282	92.00	351,087	96.48	Minimal	Very Low
Gshare	1.282	92.03	349,828	96.48	Medium (PHT+BHR)	Low (XOR+SRAM)
Tournament	**1.34**	**94.13**	374,110	89.84	High (Multi-table)	Medium (Muxing)
Perceptron (TAGE-64KB)	1.548	90.53	466,121	99.89	Very High (Weights)	High (Adder Tree)

5.4 Analysis

5.4.1 IPC and Direction Accuracy

The tournament predictor achieves the best direction accuracy (94.13%) and IPC (1.34) among the simpler predictors — a 4.5% IPC improvement over the Baseline. The adaptive combination of local and global history outperforms either sub-predictor in isolation for mixed workloads.

The Perceptron (TAGE-64KB) achieves the highest IPC (1.548) at the cost of a very large weight-matrix hardware budget and an adder-tree for the dot product, making it suited to server-class cores rather than embedded designs.

5.4.2 BTB Hit Rate

The tournament predictor shows a lower BTB hit rate (89.84%) than the others (96.48%). This occurs because the tournament predictor's higher direction accuracy allows it to

predict *not-taken* more confidently on certain branches, reducing the fraction of cases where the BTB entry is consulted and required to match. The Perceptron achieves 99.89%, reflecting its ability to confidently predict taken branches and keep the BTB warm.

5.4.3 Misprediction Count

The tournament predictor records more conditional mispredictions (374,110) than Gshare (349,828) in absolute terms, but a higher IPC means it retires more instructions per run — so the *misprediction rate per branch* is lower. The absolute count increases because the processor executes further ahead and encounters more branch instructions per benchmark run.

5.4.4 Speedup Summary

Table 5.2: Speedup of the Tournament Predictor relative to a stall-on-branch baseline

Workload	**Speedup**
Simple counting loop (Benchmark 1, highly predictable)	**5.6**×
Nested loop with unpredictable branch (Benchmark 2)	**2.0**×

The 5.6× speedup on the counting loop reflects near-perfect prediction: after one iteration the local predictor learns the 4-taken then 1-not-taken pattern and mispredicts only on the loop-exit event. The 2× speedup on the unpredictable workload demonstrates the tournament predictor's robustness: even when no predictor achieves high accuracy on the chaotic branch at PC 6, the choice predictor minimises damage by selecting whichever sub-predictor has been locally more accurate.

5.4.5 Predictor Comparison Summary

Table 5.3: Qualitative comparison of all evaluated predictors

Predictor	**Strengths**	**Weaknesses**	**Best Target**
Baseline	Zero area overhead; trivially simple.	Low accuracy on correlated or periodic branches.	Minimal embedded SoC.
Gshare	Low cost; reasonable global accuracy; XOR folding reduces aliasing in PHT.	Poor on per-branch loop patterns.	Simple controllers.
Tournament	Best accuracy/cost ratio; adapts per-branch via choice predictor.	Higher area than single-predictor designs.	General-purpose SoC / Bigger cores.
Perceptron (TAGE)	Highest accuracy; captures long-history correlations with weight vectors.	Large weight matrix; adder-tree critical path; long training.	High-performance server cores.

Chapter 6

Future Scope and Conclusion

6.1 Future Scope

6.1.1 Direct Pipeline Integration

The current design implements the BPU as a standalone module verified with unit-level testbenches. The next step is to connect `final_bpu` directly into the in-house 5-stage RISC-V processor (*RISC-RX*): wiring the branch-resolved signals from the EX/MEM pipeline register back to the BPU update ports, and feeding `next_pc` directly into the IF stage PC register with a flush-enable signal for squashing in-flight instructions on misprediction.

6.1.2 Neural Perceptron Predictor

Transitioning from 2-bit saturating counters to a **perceptron-based** predictor is the highest-impact accuracy improvement achievable within a reasonable budget. A perceptron assigns a weight vector to each GHR-indexed entry; the prediction is the sign of the dot product of that weight vector with a history vector of ± 1 values. This captures correlations across history lengths far beyond what a 12-bit GHR supports, at the cost of an adder tree for the dot product.

6.1.3 RTL-to-GDSII Physical Implementation

Synthesising the BPU using the OpenROAD/LibreLane flow targeting the SKY130 PDK (as already applied to other modules such as `tt_um_fft`) would allow direct quantification of Power, Performance, and Area (PPA) overheads at the gate level — closing the loop between the architectural accuracy numbers from gem5 and real silicon cost.

6.1.4 Extended Benchmark Suite

Future evaluation should use industry-standard RISC-V benchmark suites (SPEC CPU, CoreMark, Dhrystone) to obtain workload-independent accuracy and IPC numbers, and to study the predictor's behaviour on pointer-chasing, object-oriented dispatch, and irregular control flow.

6.2 Conclusion

This project demonstrates the complete design flow of a Tournament Branch Prediction Unit for a 5-stage RISC-V pipelined processor: from the theoretical foundations of control hazards through the architectural design of the local/global/choice predictor hierarchy and BTB with anti-aliasing logic, to a fully functional synthesisable Verilog RTL implementation (published at https://github.com/adityaamehra/Branch-Prediction-Unit) and gem5-based performance evaluation.

Key conclusions:

1. **Branch prediction is essential.** Even a 2-bit bimodal predictor eliminates most stalls on regular loop-intensive workloads; more sophisticated designs handle correlated and data-dependent branches.
2. **The tournament predictor strikes the best accuracy–cost balance.** By dynamically selecting between local and global sub-predictors via the choice predictor, it achieves 94.13% direction accuracy and an IPC of 1.34, outperforming the Baseline by 4.5%.
3. **Aliasing is a real and solvable hardware problem.** XOR-based functional addressing (PC $\oplus$ GHR) eliminates destructive aliasing in the prediction tables at negligible cost. Full-tag validation in the BTB prevents incorrect target addresses from corrupting unrelated branches.
4. **Speedup is workload-dependent:** up to 5.6× on highly predictable loop workloads and approximately 2× on unpredictable data-dependent branches — both significant over a stall-on-branch baseline.
5. **The BPU is a fundamental enabler, not merely an optimisation.** Modern processors with 15–20 stage pipelines would be entirely impractical without accurate branch prediction.